In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Get the Data

Read in the advertising.csv file and set it to a data frame called ad_data.

In [2]:
```
df = pd.read_csv('advertising (1).csv')
```

Check the head of ad_data

In [3]:
```
df
```

Out[3]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Coun
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tun
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Na
2	69.47	26	59785.94	236.50	Organic bottom-line service-desk	Davidton	0	San Mar
3	74.15	29	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	1	It
4	68.37	35	73889.99	225.58	Robust logistical utilization	South Manuel	0	Icela
...	...	...	...	...	...	...	...	
995	72.97	30	71384.57	208.58	Fundamental modular algorithm	Duffystad	1	Leban
996	51.30	45	67782.17	134.42	Grass-roots cohesive monitoring	New Darlene	1	Bosnia a Herzegov
997	51.63	51	42415.72	120.37	Expanded intangible solution	South Jessica	1	Mongo
998	55.55	19	41920.79	187.95	Proactive bandwidth-monitored policy	West Steven	0	Guatem
999	45.01	26	29875.80	178.35	Virtual 5thgeneration emulation	Ronniemouth	0	Br

1000 rows × 10 columns

** Use info and describe() on ad_data**

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Daily Time Spent on Site  1000 non-null   float64
 1   Age                       1000 non-null   int64  
 2   Area Income               1000 non-null   float64
 3   Daily Internet Usage      1000 non-null   float64
 4   Ad Topic Line             1000 non-null   object 
 5   City                      1000 non-null   object 
 6   Male                      1000 non-null   int64  
 7   Country                   1000 non-null   object 
 8   Timestamp                 1000 non-null   object 
 9   Clicked on Ad             1000 non-null   int64  
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

In [5]:
```
df.describe()
```

Out[5]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Male	Clicke
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.0
mean	65.000200	36.009000	55000.000080	180.000100	0.481000	0.5
std	15.853615	8.785562	13414.634022	43.902339	0.499889	0.5
min	32.600000	19.000000	13996.500000	104.780000	0.000000	0.0
25%	51.360000	29.000000	47031.802500	138.830000	0.000000	0.0
50%	68.215000	35.000000	57012.300000	183.130000	0.000000	0.5
75%	78.547500	42.000000	65470.635000	218.792500	1.000000	1.0
max	91.430000	61.000000	79484.800000	269.960000	1.000000	1.0

Exploratory Data Analysis

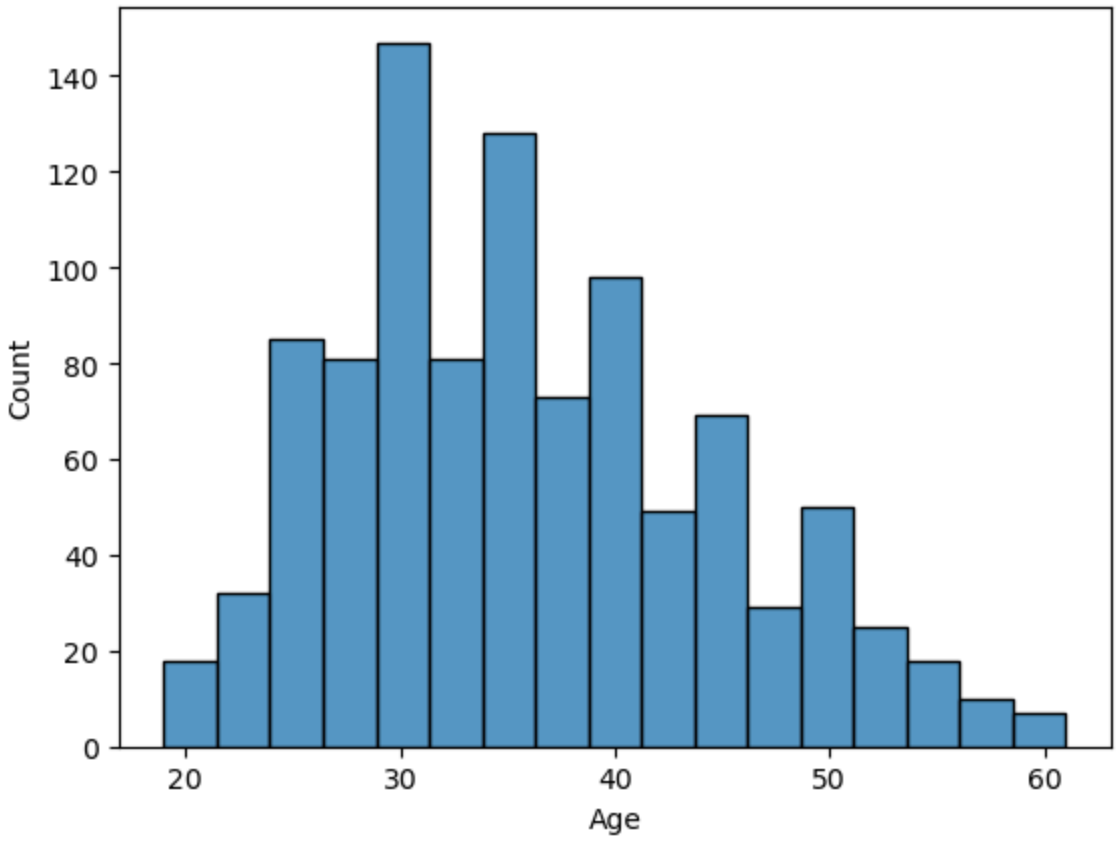
Let's use seaborn to explore the data!

Try recreating the plots shown below!

** Create a histogram of the Age**

In [7]:
```
sns.histplot(df['Age'])
```

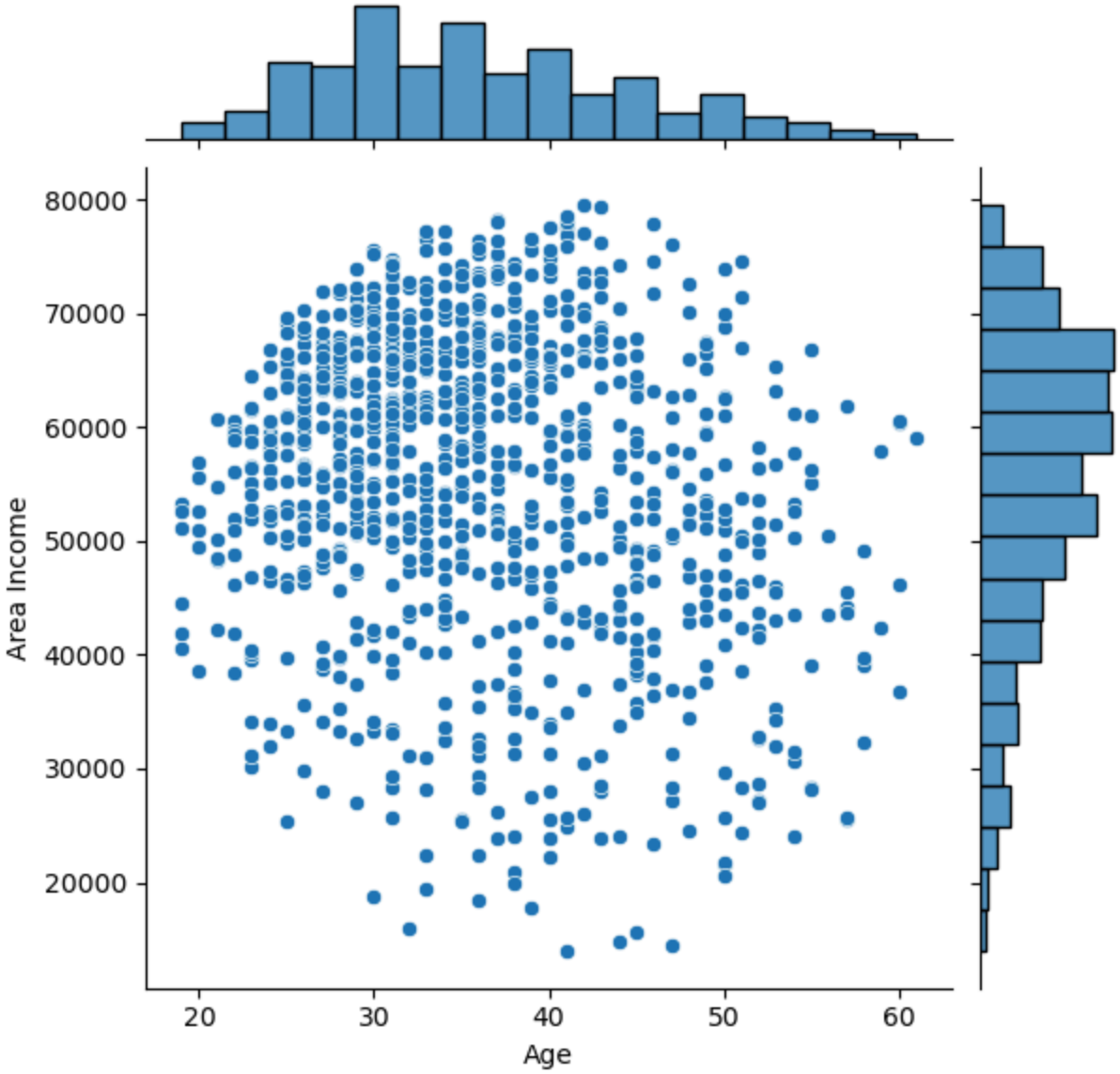
Out[7]:
```
<Axes: xlabel='Age', ylabel='Count'>
```



Create a jointplot showing Area Income versus Age.

In [8]:
```
sns.jointplot(data = df , x= 'Age',y = 'Area Income')
```

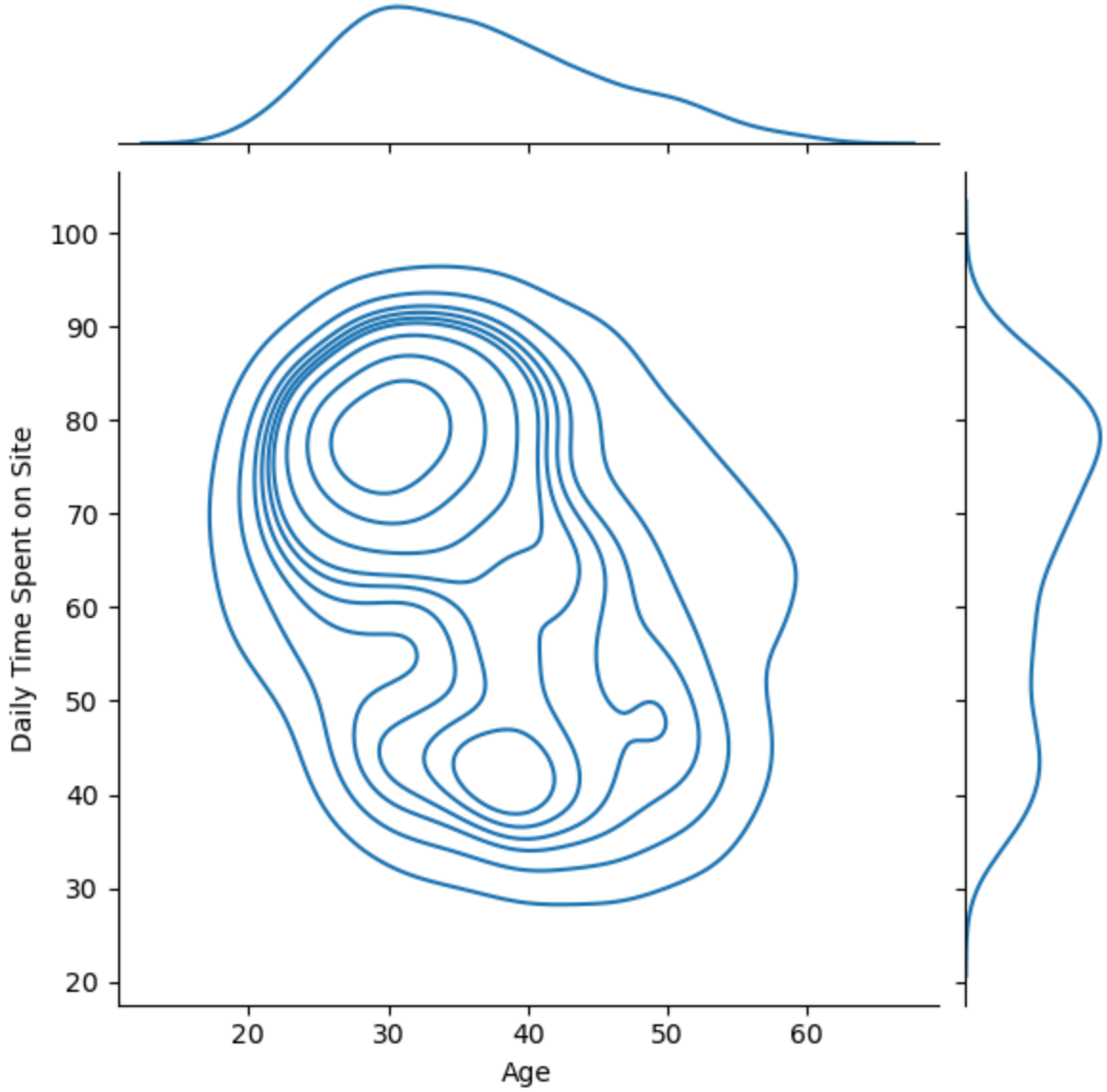
Out[8]:
```
<seaborn.axisgrid.JointGrid at 0x20d7116d430>
```



Create a jointplot showing the kde distributions of Daily Time spent on site vs. Age.

In [15]:
```
sns.jointplot(data = df , x = 'Age', y = 'Daily Time Spent on Site', kind = 'k
```

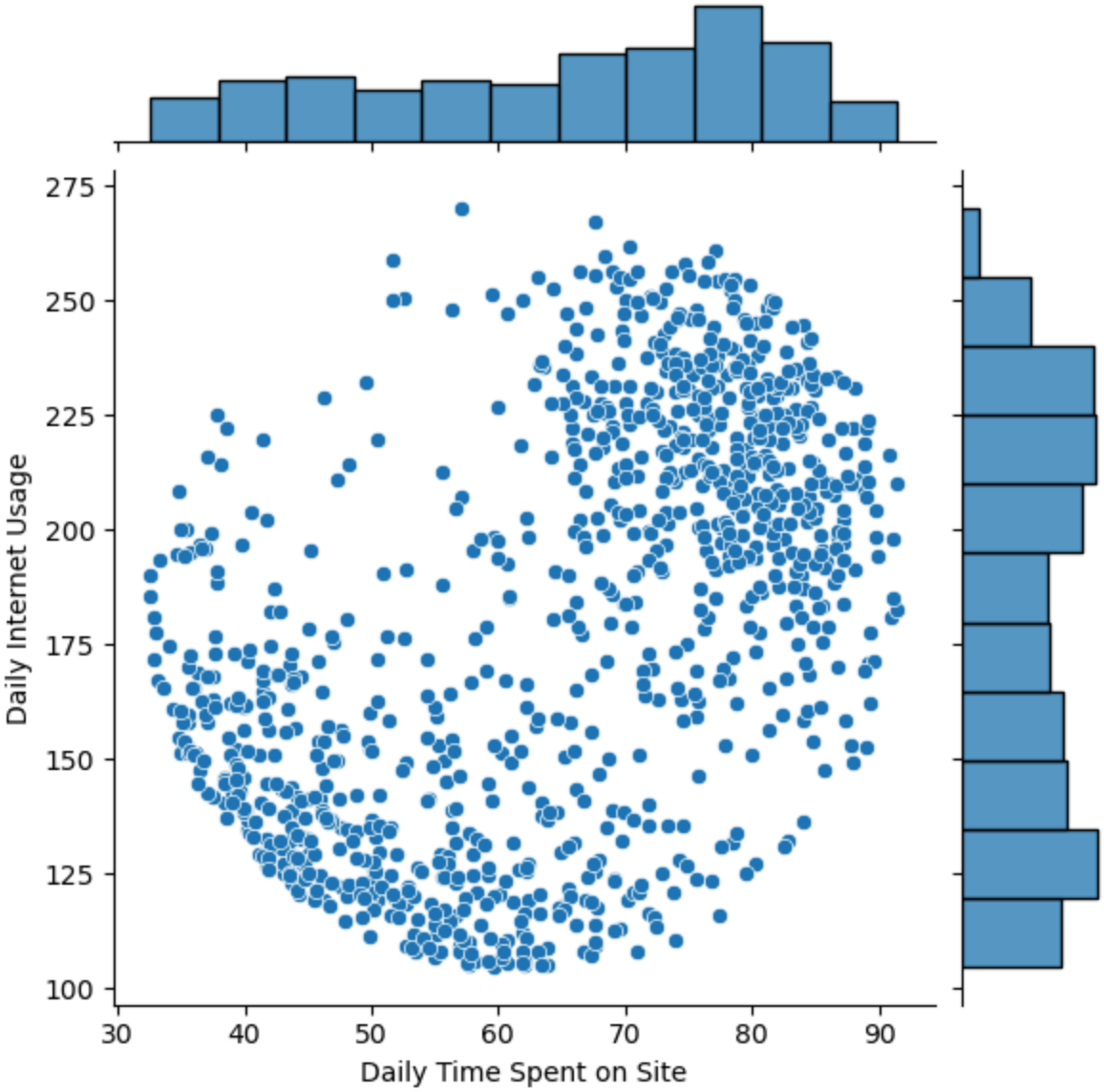
Out[15]:
```
<seaborn.axisgrid.JointGrid at 0x20d7bf5b6b0>
```



** Create a jointplot of 'Daily Time Spent on Site' vs. 'Daily Internet Usage'**

In [16]:
```
sns.jointplot(data = df , x = 'Daily Time Spent on Site', y = 'Daily Internet
```

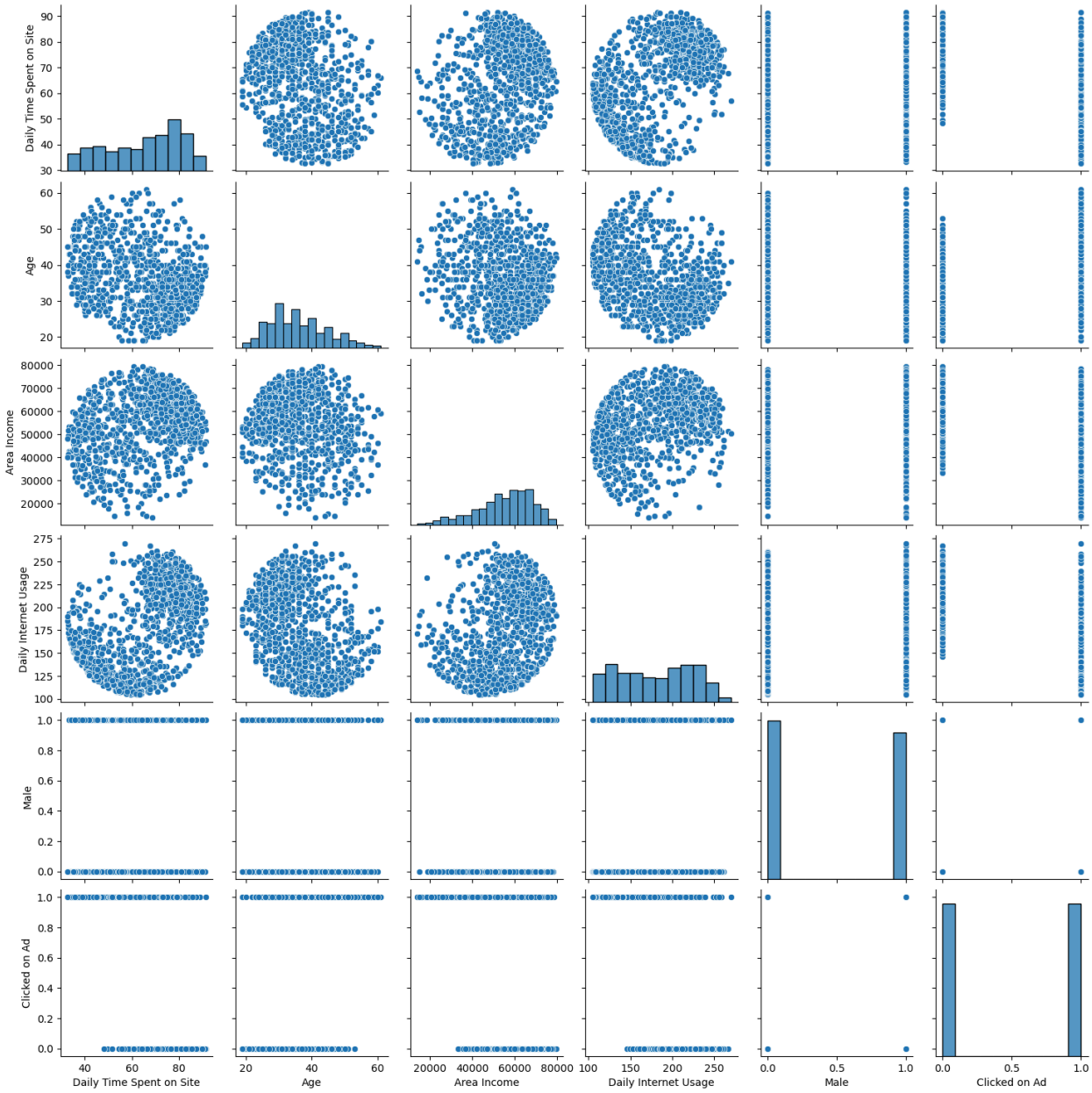
Out[16]:
```
<seaborn.axisgrid.JointGrid at 0x20d73c05e50>
```



** Finally, create a pairplot with the hue defined by the 'Clicked on Ad' column feature.**

In [17]:
```
sns.pairplot(data = df)
```

Out[17]:
```
<seaborn.axisgrid.PairGrid at 0x20d7c4c6de0>
```



Logistic Regression

Now it's time to do a train test split, and train our model!

You'll have the freedom here to choose columns that you want to train on!

** Split the data into training set and testing set using train_test_split**

In [19]:
```
from sklearn.model_selection import train_test_split
```

In [20]:
```
df.columns
```

Out[20]:
```
Index(['Daily Time Spent on Site', 'Age', 'Area Income',
       'Daily Internet Usage', 'Ad Topic Line', 'City', 'Male', 'Country',
       'Timestamp', 'Clicked on Ad'],
      dtype='object')
```

In [22]:
```
X = df[['Daily Time Spent on Site', 'Age', 'Area Income',
        'Daily Internet Usage', 'Male']]
y = df['Clicked on Ad']
```

** Train and fit a logistic regression model on the training set.**

In [23]:
```
X_train, X_test , y_train , y_test = train_test_split(X,y,test_size = 0.3, ran
```

In [25]:
```
from sklearn.linear_model import LogisticRegression
```

In [26]:
```
log = LogisticRegression()
```

In [29]:
```
log.fit(X_train,y_train)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\linear_model\_logistic.py:46
9: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regressi
on
  n_iter_i = _check_optimize_result(
```

Out[29]:
```
LogisticRegression
LogisticRegression()
```

Predictions and Evaluations

** Now predict values for the testing data.**

In [30]:
```
prediction = log.predict(X_test)
```

** Create a classification report for the model.**

In [31]:
```
from sklearn.metrics import classification_report
```

In [33]:
```
print(classification_report(y_test, prediction))
```

	precision	recall	f1-score	support
0	0.91	0.95	0.93	157
1	0.94	0.90	0.92	143
accuracy			0.93	300
macro avg	0.93	0.93	0.93	300
weighted avg	0.93	0.93	0.93	300

Great Job!